

计算机设计与实践

单周期CPU设计

基于Trace的CPU功能验证

2026 · 夏



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

实验目的

- ◆ 理解基于Trace的CPU功能验证方法的基本原理
- ◆ 掌握如何使用Trace验证方法



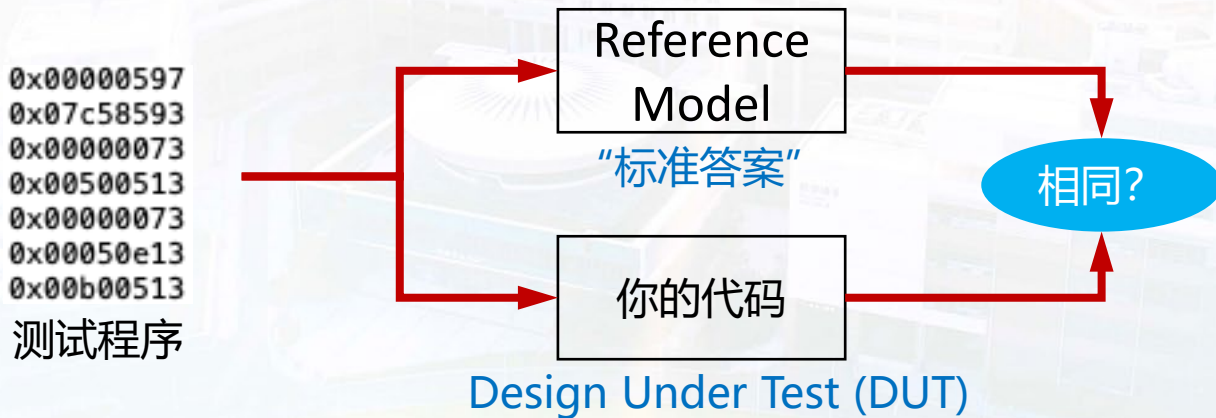
实验内容

- ◆ 了解Trace验证原理
- ◆ 理解Trace测试所需连接的调试信号的含义
- ◆ 在虚拟机中运行Trace测试，结合波形、反汇编、HDL代码进行调试



Trace比对

- ◆ Trace: CPU执行指令序列时产生的信息 (PC、**写寄存器**的信息, etc)
- ◆ 基于Trace比对的验证方法:
 - ① 用已知功能正确的CPU运行测试程序, 记录Trace0 (Golden Trace)
 - ② 用待验证CPU运行测试程序, 产生Trace1
 - ③ 将Trace1与Trace0进行实时比对, 如果出现不同, 立即报错并停止



Trace比对

- ◆ Trace比对的核心：检查写寄存器、写访存的信息是否正确
 - ◆ 只有当debug_wb_rf_we有效时，才会触发写寄存器的比对
 - ◆ 只有当debug_mem_pc有效时，才会触发写访存的比对

```
01 wire [31:0] debug_wb_pc    ;    // WB阶段的PC
02 wire          debug_wb_rf_we;    // WB阶段的寄存器写使能
03 wire [ 4:0] debug_wb_rf_wR;    // WB阶段的目标寄存器    (若wb_rf_we为0, 此项可为任意值)
04 wire [31:0] debug_wb_rf_wD;    // WB阶段写入寄存器的值  (若wb_rf_we为0, 此项可为任意值)
05
06 wire [31:0] debug_mem_pc    ;    // MEM阶段的PC
07 wire [ 3:0] debug_mem_we    ;    // MEM阶段写访存时的写使能
08 wire [31:0] debug_mem_waddr;    // MEM阶段写访存时的写地址 (若mem_we为0, 此项可为任意值)
09 wire [31:0] debug_mem_wdata;    // MEM阶段写访存时的写数据 (若mem_we为0, 此项可为任意值)
```

注意：Trace框架只会验证CPU发出的写访存请求是否正确

若写访存请求正确，但DCache或总线有bug，则写访存仍会出错



Trace比对 – 要求

- ◆ Trace框架提供的fpga_rst是高电平复位
- ◆ 只拷贝src/rtl目录下的源文件，不要拷贝IP核
 - ◆ 拷贝代码到WSL可能生成 Zone.Identifier 文件，需要删除之
- ◆ 不要修改现有代码的IROM、DRAM的模块名称
- ◆ 不要修改出现RUN_TRACE宏定义的代码
- ◆ 不要修改出现 `/* verilator public */` 的代码
- ◆ PC初始值：0x0000_0000
 - ◆ 复位后CPU执行的首条指令的地址是0x0000_0000

Trace比对 – 使用

◆ Trace使用

- ◆ 环境：推荐直接用实验室安装好的WSL虚拟机
- ◆ 使用：git clone下载Trace测试包，将SoC源码拷贝至mySoC目录，在cdp-tests目录输入命令make即可
- ◆ 出现问题，执行make clean后重试

◆ 详细说明：

Trace测试说明

测试机制说明

附录1：远程平台使用指南

附录2：虚拟机使用指南

附录3：实验环境部署指南

附录2：虚拟机使用指南

1. 下载虚拟机

测试环境及相关工具下载地址：

- 虚拟机镜像：[vm.ova](#)
- MobaXTerm和VirtualBox：[虚拟机工具.zip](#)

下载后，请参照本页后的“附录”导入虚拟机。



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ